

DFS & Components

Graphs, Depth-First Search, Connected Components · Mentor-Led Lab
(Student Edition)

Developed by TalenciaGlobal

Difficulty ★★★ Advanced

Problem

Run a depth-first search over the whole graph and count its connected components. You are given starter code with two functions. `dfs_all` is almost complete but has a bug, and `count_components` is an empty stub. Fix the bug and implement the stub.

The DFS order must cover every vertex, restarting from each unvisited vertex in increasing order.

What to Compute

- DFS Order: the depth-first visiting order over the whole graph.
- Components: the number of connected components.

Input / Output Format

Input: Line 1: N M . Next M lines: an undirected edge u v .

Output: Two lines: DFS Order and Components.

Examples

Example 1

Input: 6 3 0 1 1 2 3 4

Output:

DFS Order: 0 1 2 3 4 5
Components: 3

Explanation: Component $\{0,1,2\}$, then $\{3,4\}$, then the isolated $\{5\}$; the DFS restarts from each unvisited vertex, giving three components.

Example 2

Input: 4 3 0 1 0 2 0 3

Output:

DFS Order: 0 1 2 3
Components: 1

Explanation: A star centred at 0: one component, and DFS visits 0 then its sorted neighbours.

Constraints

- DFS explores neighbours in sorted order.
- Start a new DFS from each unvisited vertex in increasing order.
- Each restart marks a new component.

Your Task

- Task 1 - Fix the bug. `dfs_all` is flagged with a comment, but the bug's location is not given. Find the single bug so the DFS visits EVERY vertex (restarting from each unvisited one).
- Task 2 - Implement `count_components` from scratch: the number of connected components.

Starter Code

Begin from the code below. One function carries a single bug (flagged by a comment, with no hint to its location); the other is an empty stub you must implement.

Python

```
import sys
from collections import deque

def read_ints():
    return [int(x) for x in sys.stdin.read().split()]

def build_graph(data):
    n = data[0]
    m = data[1]
    adj = [set() for _ in range(n)]
    idx = 2
    for _ in range(m):
        u = data[idx]
        v = data[idx + 1]
        idx += 2
        adj[u].add(v)
        adj[v].add(u)
    adj = [sorted(s) for s in adj]
    return n, m, adj, idx

def dfs(u, adj, visited, order):
    visited[u] = True
    order.append(u)
    for w in adj[u]:
        if not visited[w]:
            dfs(w, adj, visited, order)
```

```

# BUG: this function contains a single bug. It must visit EVERY vertex, continuing
# to each unvisited vertex so that all connected components appear in the order.
def dfs_all(n, adj):
    visited = [False] * n
    order = []
    dfs(0, adj, visited, order)
    return order

# TODO: implement this. It must count the number of connected components.
# Currently a stub.
def count_components(n, adj):
    return 0

def main():
    data = read_ints()
    n, m, adj, idx = build_graph(data)
    order = dfs_all(n, adj)
    print("DFS Order: " + " ".join(str(x) for x in order))
    print("Components: " + str(count_components(n, adj)))

main()

```

C++

```

// Graph Lab 3 - DFS & Components (STARTER)
#include <iostream>
#include <vector>
#include <string>
#include <set>
#include <queue>
#include <algorithm>
using namespace std;

vector<long long> readInts() {
    vector<long long> data;
    long long x;
    while (cin >> x) {
        data.push_back(x);
    }
    return data;
}

vector<vector<int>> buildGraph(const vector<long long>& data, int& n, int& m, int& idx) {
    n = (int) data[0];
    m = (int) data[1];
    vector<set<int>> tmp(n);
    idx = 2;
    for (int i = 0; i < m; i++) {
        int u = (int) data[idx];
        int v = (int) data[idx + 1];
        idx += 2;
        tmp[u].insert(v);
        tmp[v].insert(u);
    }
    vector<vector<int>> adj(n);
    for (int i = 0; i < n; i++) {

```

```

        adj[i] = vector<int>(tmp[i].begin(), tmp[i].end());
    }
    return adj;
}

void dfs(int u, const vector<vector<int>>& adj, vector<bool>& visited, vector<int>& order) {
    visited[u] = true;
    order.push_back(u);
    for (int w : adj[u]) {
        if (!visited[w]) {
            dfs(w, adj, visited, order);
        }
    }
}

// BUG: this function contains a single bug. It must visit EVERY vertex, continuing
// to each unvisited vertex so that all connected components appear in the order.
vector<int> dfsAll(int n, const vector<vector<int>>& adj) {
    vector<bool> visited(n, false);
    vector<int> order;
    dfs(0, adj, visited, order);
    return order;
}

// TODO: implement this. It must count the number of connected components.
// Currently a stub.
int countComponents(int n, const vector<vector<int>>& adj) {
    return 0;
}

int main() {
    vector<long long> data = readInts();
    int n, m, idx;
    vector<vector<int>> adj = buildGraph(data, n, m, idx);
    vector<int> order = dfsAll(n, adj);
    cout << "DFS Order:";
    for (int x : order) {
        cout << " " << x;
    }
    cout << "\n";
    cout << "Components: " << countComponents(n, adj) << "\n";
    return 0;
}

```

Java

```

// Graph Lab 3 - DFS & Components (STARTER)
import java.util.*;

public class Main {
    static int N, M;

    static List<List<Integer>> buildGraph(Scanner sc) {
        List<Long> data = new ArrayList<>();
        while (sc.hasNextLong()) {
            data.add(sc.nextLong());
        }
    }
}

```

```

    DATA = data;
    N = (int) (long) data.get(0);
    M = (int) (long) data.get(1);
    List<TreeSet<Integer>> tmp = new ArrayList<>();
    for (int i = 0; i < N; i++) {
        tmp.add(new TreeSet<>());
    }
    int idx = 2;
    for (int i = 0; i < M; i++) {
        int u = (int) (long) data.get(idx);
        int v = (int) (long) data.get(idx + 1);
        idx += 2;
        tmp.get(u).add(v);
        tmp.get(v).add(u);
    }
    EDGE_END = idx;
    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < N; i++) {
        adj.add(new ArrayList<>(tmp.get(i)));
    }
    return adj;
}

static List<Long> DATA;
static int EDGE_END;

static void dfs(int u, List<List<Integer>> adj, boolean[] visited, List<Integer> order) {
    visited[u] = true;
    order.add(u);
    for (int w : adj.get(u)) {
        if (!visited[w]) {
            dfs(w, adj, visited, order);
        }
    }
}

// BUG: this function contains a single bug. It must visit EVERY vertex, continuing
// to each unvisited vertex so that all connected components appear in the order.
static List<Integer> dfsAll(int n, List<List<Integer>> adj) {
    boolean[] visited = new boolean[n];
    List<Integer> order = new ArrayList<>();
    dfs(0, adj, visited, order);
    return order;
}

// TODO: implement this. It must count the number of connected components.
// Currently a stub.
static int countComponents(int n, List<List<Integer>> adj) {
    return 0;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    List<List<Integer>> adj = buildGraph(sc);
    List<Integer> order = dfsAll(N, adj);
    StringBuilder sb = new StringBuilder("DFS Order:");
    for (int x : order) {

```

```
        sb.append(" ").append(x);
    }
    sb.append("\n");
    sb.append("Components: ").append(countComponents(N, adj)).append("\n");
    System.out.print(sb);
}
}
```

Sample Run

Sample Input

```
6 3
0 1
1 2
3 4
```

Sample Output

```
DFS Order: 0 1 2 3 4 5
Components: 3
```